

Health Hub

Complete Project Documentation

Tech stack • Architecture • Features • Data flows

Author: Kundan Krishna
Institution: Sathyabama Institute of Science and Technology, Chennai
Live URL: <https://kundankrishna.tech/healthhub/>
Repository: <https://github.com/kundankrishna001/HEALTHCHECK26>
Generated for presentations, viva, and technical walkthroughs.

1. Executive Summary

Health Hub is a full-stack HealthTech web application that helps users track wellness habits, check symptoms, plan meals, scan food, and manage family health profiles from a single dashboard. The frontend is a React single-page application (SPA) served by Nginx. The backend is a Node.js Express REST API with JWT authentication and MySQL persistence. AI-powered guidance uses a resilient fallback knowledge base stored in MySQL rather than a live external LLM, ensuring the app works even when AI services are unavailable.

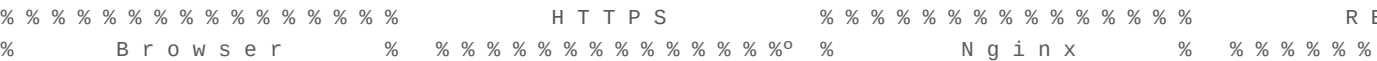
2. Problem Statement & Purpose

- People struggle to track nutrition, hydration, symptoms, and health metrics in one place.
- Health Hub centralizes symptom checking, diet planning, recipes, water tracking, and family profiles.
- Designed for students, families, and everyday users who want practical wellness tools.
- Domain: HealthTech & Wellness — personal health management and habit tracking.

3. Technology Stack

Layer	Technologies
Languages	JavaScript (ES modules), HTML, CSS, SQL
Frontend	React 18, React Router 6, Vite 5, Chart.js, react-hook-form, Yup
Backend	Node.js, Express 4, JWT, bcryptjs, CORS, dotenv
Database	MySQL 8 — users, user_states (JSON), ai_fallback_knowledge
PDF export	jsPDF (client-side health data export)
Deployment	DigitalOcean droplet, Nginx, PM2/systemd, Certbot (HTTPS)
Dev tooling	npm, concurrently, Python deploy script, Bash/PowerShell scripts

4. High-Level System Architecture



5. Frontend Architecture

- Entry: index.html! 'main.jsx!' App.jsx with React Router (basename /health)
- Context providers: ThemeContext, AuthContext, AppContext wrap the entire app.
- Public routes: /, /features, /about, /login, /signup, /forgot-password, /reset-password.
- Protected routes: /app/* wrapped by ProtectedRoute + AppShell (sidebar, topbar, mobile nav).
- State: AppContext loads user state once via GET /api/state; mutations use PUT /api/state.
- Services: authService, stateService, httpClient, aiEngine, demoMode, exportPdf.

6. Backend API Endpoints

Method	Endpoint	Auth	Purpose
GET	/api/health	No	API health check
POST	/api/auth/signup	No	Create account + default state
POST	/api/auth/login	No	Email/password login! JWT
POST	/api/auth/demo	No	Demo user session
GET	/api/auth/me	Yes	Current user profile
POST	/api/auth/logout	Yes	Logout (stateless)
POST	/api/auth/reset-request	No	Password reset token
POST	/api/auth/reset-password	No	Set new password
GET	/api/state	Yes	Load or create user JSON state
PUT	/api/state	Yes	Deep-merge and save state
POST	/api/ai-fallback	Yes	Knowledge-base health
GET	/api/admin/diagnostics	Admin	DB and state diagnostics

7. Database Schema

users
id, name, email, password_hash, role, reset_token, reset_token_expires, timestamps

user_states
user_id (FK), state_json (JSON blob), timestamps

ai_fallback_knowledge
id, type, input_keyword, output_json, created_at
Types: symptoms | weeklyPlan | recipe | checkFood | scanFood

User state JSON contains: user profile, family[], symptoms[], meals[], water[], metrics[], recipes[], dietPlans[], badges[], settings{}

8. Authentication Flow

1. User opens app! 'AuthContext' reads JWT from browser sto

```
2. If token exists !' GET /api/auth/me validates session
3. If no token !' user sees public pages (Welcome, Login)
4. Signup/Login !' POST /api/auth/* !' JWT stored !' redirect
5. ProtectedRoute blocks /app/* without valid user
6. httpClient attaches Bearer token; 401 clears token and redirects to login
7. Demo mode: POST /api/auth/demo OR offline localStorage fallback
```

9. Application Data Flow

```
User action (e.g. log water)
    %
    %¼
React page component
    %
    %¼
AppContext !' stateService.logWater()
    %
    %¼
GET current state !' merge change !' PUT /api/state
    %
    %¼
MySQL user_states.state_json updated
    %
    %¼
AppContext refreshes local state !' UI re-renders
```

10. AI / Health Guidance Flow

The aiEngine simulates AI failure by design, then queries the fallback API:

```
Symptom / Recipe / Food request
    %
    %¼
aiEngine.js (client)
    %
    %¼
POST /api/ai-fallback { type, query, profile }
    %
    %¼
MySQL ai_fallback_knowledge lookup by keyword + profile preference
    %
    %¼
JSON response !' rendered in UI (guidance, meal plan, recipe)
    %
    %¼
If API fails (offline demo) !' hardcoded client defaults in
```

11. Feature Modules (18 modules)

- Dashboard: Wellness score, hydration, nutrition tiles, charts, quick actions
- Profile: Name, age, height, weight, conditions, goals, auto BMI
- Symptom Checker: Natural-language symptoms !' food guidance + diet plan
- Diet Plan: 7-day meal generator, save/load plans, shopping list
- Recipes: Generate, save, favorite, print, share recipes
- Food Scanner: Text/barcode/image-hint food analysis
- Food Checker: Safety, quantity, medication interactions
- Nutrition: Meal logging, calorie charts, weekly plans, PDF export
- Water Tracker: Daily hydration with quick-add (200–750 ml)

- Health Metrics: Weight, steps, sleep, vitals, personal goals
- Family: Multiple profiles, active switcher, per-member stats
- Achievements: Points, streaks, badges (client-computed)
- Doctor Reports: Report upload UI with sample parsing (demo)
- Corporate: Employee wellness dashboard (static demo)
- School: Tiffin planner and menu suggestions (static demo)
- Settings: Dark mode, notifications, reminders, PDF data export
- Language: 9 Indian languages + browser voice input
- Help & Privacy: FAQ and data-handling information

12. User Journey Flow

```

Landing ( / ) !'  Features ( / features ) or Sign in ( / login )
%
% %   Try Demo !' / app Dashboard ( instant access )
% %   Sign up !' / app / profile !' complete profile !' / a p
% %   Sign in !' / app Dashboard
%
% %   Log water / meals / metrics
% %   Run symptom or food checks
% %   Generate diet plans & recipes
% %   Export PDF from Settings

```

13. Deployment Architecture

- Build: `npm run build` !' static files in `dist/`
- Server path: `/opt/health-hub` (API), `/var/www/.../healthhub` (static)
- Nginx: serves SPA at `/healthhub/` and proxies `/healthhub/api/` to Express
- Process manager: `health-hub-api` systemd service on port 4000
- SSL: Certbot Let's Encrypt for `kundankrishna.tech`
- Deploy script: `deploy/remote_deploy.py` (build + rsync + restart)

14. Security Measures

- Passwords hashed with bcrypt (12 rounds).
- JWT tokens (7-day expiry) for authenticated API calls.
- Protected routes require valid Bearer token.
- Reset tokens expire after 1 hour; generic messages prevent email enumeration.
- Secrets in `server.env` — not committed to Git.
- HTTPS enforced in production via Certbot.

15. Demo Mode

- Server demo: `POST /api/auth/demo` creates/uses `demo@example.com` with full API access.
- Offline demo: `localStorage` token `demo-local-session` with pre-seeded state.
- Offline demo skips API calls; AI uses inline hardcoded fallbacks.

16. Interface Languages Supported

English, Hindi, Tamil, Telugu, Bengali, Gujarati, Marathi, Punjabi, Kannada — selectable on Language page with Web Speech API voice input.

17. Known Limitations

- Password reset generates a token but does not send email in production yet.
- Doctor/Corporate/School modules use demo/static data.
- AI responses come from MySQL fallback, not a live LLM.
- Language selection does not fully translate the entire UI.

18. How to Run Locally

- `npm install`
- Copy `.env.example` and `server/.env.example`; set MySQL + JWT_SECRET.
- `npm run dev:all` — frontend `http://localhost:5173`, API `http://localhost:4000`
- `npm run build` — production static build

19. Presentation Talking Points

- Explain the three-tier architecture: React ! Express ! MySQL.
- Show live demo: login ! dashboard ! symptom check ! save diet plan.
- Highlight JWT auth and JSON state blob design for flexible health data.
- Describe fallback AI pattern for reliability without external API dependency.
- Mention deployment on DigitalOcean with path-based hosting (/healthhub).